Accuracy, Interpretability and Practicality in Bitcoin Volatility Forecasting: Machine Learning versus Statistical Models

Abstract

This project investigates whether the additional complexity of machine-learning models is justified when forecasting Bitcoin volatility. Five models are evaluated on Hyperliquid BTC perpetual-futures daily data: rolling historical volatility, GARCH(1,1), lagged linear regression, Random Forest and a Long Short-Term Memory (LSTM) network. The target is the next day's updated standard deviation of log returns, calculated primarily over 30 days. A fixed chronological cutoff supplies the primary test, while a 14-day target, four expanding-window folds, two test-period segments, volatility regimes, a moving-block bootstrap and a later seven-day data refresh provide robustness evidence. Accuracy is considered alongside interpretability, measured local runtime, structural complexity and reproducibility.

GARCH(1,1) records the lowest primary RMSE, `0.00098502`, which is 30.994% below rolling historical volatility. It also ranks first for the 14-day target, both test halves, all expanding-window folds and all three target-volatility regimes. Its paired moving-block-bootstrap interval for the RMSE difference from rolling is `[-0.00099670, -0.00017158]`. Random Forest is worse than rolling overall, while LSTM is locally competitive in the first and fourth expanding-window folds but does not produce a stable aggregate improvement. The conclusion is deliberately bounded: under this dataset, target and implementation, added machine-learning complexity is not justified by a sufficiently clear and robust accuracy gain.

1. Introduction

Volatility describes the scale of variation in returns rather than the direction of price movement. A price-direction forecast asks whether Bitcoin will rise or fall; a volatility forecast asks how uncertain or variable the next period may be. Traders and risk managers can use such an estimate when setting position sizes, risk limits or collateral buffers even when they do not know the next return's sign. Cryptocurrency is useful for this investigation because sharp movements and persistent market stress make changing uncertainty visible, while public exchange data make an independent comparison reproducible.

The project asks:

> How do Random Forest and Long Short-Term Memory networks compare with rolling historical volatility and GARCH(1,1) when forecasting Bitcoin volatility, in terms of accuracy, interpretability, computational practicality and robustness?

The wording is comparative rather than promotional. Machine learning can represent nonlinear relationships and complex sequences, but flexibility is not automatically useful. It can increase overfitting risk, require more choices and make forecasts harder to explain. Statistical models may be less flexible, yet volatility persistence gives them a strong structure to exploit. The project therefore applies a demanding decision rule: a more complex method should not be preferred merely because it ranks slightly higher once. It should show a material primary error reduction, retain that advantage across chronological checks and provide enough practical or explanatory benefit to justify its additional structure.

The study constructs an explicit target from exchange candles, fits two machine-learning approaches and transparent statistical alternatives, and evaluates later observations without random shuffling. Lagged linear regression is an auxiliary comparator: it is not one of the four models named in the question, but it tests whether engineered features contain information that can be used without a nonlinear learner. The asset is specifically Hyperliquid BTC perpetual futures, not a generic Bitcoin spot index. This improves source specificity but limits generalisation to other exchanges and spot markets.

The evaluation date is fixed rather than recalculated as a moving 80/20 boundary after each download. 16 November 2025 remains the first forecast-origin date in the primary test whenever new completed candles are appended. This matters because otherwise a refresh would move several observations from test into training, changing both the evidence and the question. Applying the final code to data available on 14 and 20 July 2026 therefore tests whether seven newly completed days change the result under the same historical cutoff.

The project also treats implementation audit as evidence. Forecast dates, GARCH recursion, rolling-target conversion, incomplete candles, feature duplication and scaling boundaries were explicitly checked. The final conclusion is based on the post-audit pipeline rather than preserved merely for consistency with an earlier draft.

2. Literature review

2.1 Why GARCH remains a serious benchmark

Financial return variance is not usually constant through time. Large movements often cluster together, as do calm periods. Bollerslev (1986) developed Generalised Autoregressive Conditional Heteroskedasticity so that conditional variance could depend on recent squared shocks and earlier conditional variance. GARCH(1,1) is therefore designed around persistence rather than being a generic regression applied to a time series.

Hansen and Lunde (2005) demonstrate why GARCH(1,1) should not be treated as a weak benchmark. They compare 330 ARCH-type models out of sample. For exchange-rate data they find no evidence that more sophisticated models outperform GARCH(1,1), although asymmetric models do better for IBM returns. The lesson is conditional: GARCH can be difficult to beat in one setting and inadequate in another. Katsiampa (2017) similarly supports conditional-variance modelling for Bitcoin, finding an autoregressive-component GARCH model strongest among several GARCH-family specifications tested in sample. Neither study establishes that simple GARCH must win this experiment; together they justify treating it as a serious competitor.

2.2 Mixed evidence on machine learning

The machine-learning literature does not give one stable ranking. Dudek et al. (2024) compare 12 statistical and machine-learning methods, including GARCH, Random Forest and LSTM, on four cryptocurrencies. Performance varies with asset, loss function and horizon, and simple linear methods can match the more complex learners. Their realised variance is constructed from intraday returns, however, and is richer than the daily rolling proxy used here.

Huang, Sangiorgi and Urquhart (2024) provide a contrasting result. Using high-frequency Bitcoin data from 2014 to 2021, they report neural-network improvements over GARCH across their tested horizons. Their LSTM and CNN-LSTM models receive richer transformations and many more intraday observations, so a small daily-data LSTM is not an equivalent replication. Shen, Wan and Leatham (2021) also find that a recurrent network performs better on average forecasting measures, but less effectively for extreme events and Value at Risk. That distinction supports evaluating high-volatility behaviour rather than relying only on aggregate RMSE.

Hybrid research weakens the idea that statistical and machine-learning methods are opposing camps. Zahid, Iqbal and Koutmos (2022) combine GARCH structures with machine learning for Bitcoin realised-volatility forecasting. A hybrid is a plausible extension, but implementing it before establishing separate baselines would make the source of improvement harder to identify. Catania, Grassi and Ravazzolo (2019) further show that cryptocurrency predictability is affected by model and parameter instability. This project responds with a fixed chronological holdout, two time segments and four expanding-window blocks. Those checks are stronger than one split, although they still come from one market history rather than independent external samples.

2.3 Features, sequences and interpretability

Breiman (2001) describes Random Forest as an ensemble of randomised decision trees. Bootstrap sampling and random feature selection reduce dependence among trees, while averaging reduces variance. The method can capture thresholds and interactions without assuming a linear equation. In time-series work, however, it does not know chronological order automatically; past information must be represented through lags and rolling features. Feature design is therefore part of the model, not neutral preprocessing.

Hochreiter and Schmidhuber (1997) introduced LSTM to address the difficulty recurrent neural networks face when learning longer dependencies. Its gates control what information is retained, updated and exposed. That architecture appears suitable for persistent volatility, but suitability is not proof of accuracy. Sequence length, hidden size, scaling, learning rate, regularisation and stopping all matter, especially with only about one thousand training observations.

Interpretability must also be defined carefully. Lundberg and Lee (2017) show how attribution methods can explain parts of complex predictions, while Molnar (2025) stresses that different explanation tools answer different questions. This project does not claim a complete local explanation of the LSTM. It records only evidence actually produced: equations and parameters for GARCH, signed standardised coefficients for the linear model, impurity and permutation importance plus out-of-bag diagnostics for Random Forest, and architecture, losses and seed stability for LSTM. Feature importance remains associational, can be divided among correlated inputs and does not explain the direction of an individual forecast.

2.4 Volatility is measured through a proxy

True conditional volatility is latent. Patton (2011) shows that rankings can depend on the proxy and loss function used to compare forecasts. High-frequency realised variance is often more informative than a rolling standard deviation based on one daily close. The present target is consequently described as a *daily realised-volatility proxy*, not Bitcoin's true volatility.

The proxy has an important structural property: 29 of the 30 returns in tomorrow's primary window are already known today. Persistence is therefore partly mathematical as well as empirical. Rolling volatility is a demanding operational benchmark, and a model must estimate the effect of the entering return and the leaving return more accurately to beat it. This design is appropriate for an updated one-day-ahead risk indicator, but it is different from predicting a wholly future, non-overlapping 30-day window.

Overall, the literature supports the comparison without predetermining its answer. GARCH has a strong theoretical and empirical basis; machine learning can outperform it with richer data and design; simple methods can remain competitive; and rankings depend on the market, target and evaluation procedure. The present study addresses that conditional question with a small, auditable daily-data experiment.

Three expectations follow, but none is treated as a guaranteed result. Rolling volatility should be difficult to beat because of target overlap. GARCH should benefit if conditional shock information improves the estimate of the one unknown entering return. Random Forest and LSTM should benefit only if nonlinear feature interactions or sequence patterns contain information beyond that persistence. This framing prevents an after-the-fact claim that whichever model wins was always theoretically destined to do so.

3. Mathematical formulation

Let P_t be the daily closing price on day t. The logarithmic return is

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right).$$

For a window of n days, the daily realised-volatility proxy is the sample standard deviation

$$RV_t^{(n)} = \sqrt{\frac{1}{n-1}\sum_{i=t-n+1}^{t}(r_i - \bar{r}_t)^2}.$$

The primary window is $n = 30$, while $n = 14$ is a robustness check. Values are not annualised. A row formed at day t predicts

$$y_t = RV_{t+1}^{(n)},$$

and rolling historical volatility sets $\widehat{y}_t = RV_t^{(n)}$.

Feature scaling is fitted only on the relevant training portion:

$$z_{tj} = \frac{x_{tj} - \mu_{j,\text{train}}}{\sigma_{j,\text{train}}}.$$

The lagged linear model then solves a lightly regularised objective,

$$\widehat{\beta} = \arg\min_{\beta} \sum_{t\in\text{train}} (y_t - z_t^\top \beta)^2 + \lambda\|\beta\|_2^2,$$

where $\lambda = 10^{-8}$ is used for numerical stability rather than strong shrinkage. Random Forest averages B fitted trees,

$$\hat{y}_t^{RF} = \frac{1}{B}\sum_{b=1}^{B} T_b(z_t).$$

For LSTM, input and previous hidden state determine gates such as $f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$; the cell is updated by $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, and the final hidden representation is mapped to one volatility estimate. These equations describe information flow, not a direct economic interpretation of each learned weight.

GARCH(1,1) models one-step conditional variance as

$$h_{t+1} = \omega + \alpha\epsilon_t^2 + \beta h_t.$$

Here α measures shock response and β persistence. The final estimates are $\alpha = 0.10$, $\beta = 0.78$ and $\alpha + \beta = 0.88$. The grid search selects α and β; variance targeting implies $\omega = (1 - \alpha - \beta)\hat{\sigma}_r^2$, so the three reported parameters are not three independently searched quantities.

To map conditional variance to the rolling target, let S_t and Q_t be the sum and sum of squares of the known preceding $n - 1$ returns. Under zero conditional mean,

$$E[s_{t+1}^2 \mid \mathcal{F}_t] = \frac{Q_t + h_{t+1} - (S_t^2 + h_{t+1})/n}{n-1}.$$

Because the target is a standard deviation and squared-error evaluation is used, the primary implementation estimates $E[s \mid \mathcal{F}_t]$ with deterministic 80-point Gauss-Hermite quadrature. The analytic $\sqrt{E[s^2 \mid \mathcal{F}_t]}$ is retained as a sensitivity because the two expressions are not identical when the square root is concave.

For m test observations,

$$MAE = \frac{1}{m}\sum_{t=1}^{m} |y_t - \hat{y}_t|, \qquad RMSE = \sqrt{\frac{1}{m}\sum_{t=1}^{m}(y_t - \hat{y}_t)^2}.$$

RMSE gives more influence to large misses; MAE represents a typical absolute miss. Bootstrap replicate b compares each model with rolling using

$$\Delta_b = RMSE_b(M) - RMSE_b(\text{Rolling}),$$

so negative values favour the competing model.

4. Methodology

4.1 Data collection, quality and refresh control

Hyperliquid's public `candleSnapshot` endpoint supplied daily OHLCV candles (Hyperliquid, 2026). Of 1,241 returned rows, the one still-open candle was rejected using its end timestamp. The accepted archive contains 1,240 completed candles from 26 February 2023 through 19 July 2026 (`2026-07-19`).

The quality gate checks schema, timestamp order and uniqueness, daily cadence, symbol, interval, positive prices, OHLC ordering and non-negative activity. No critical issue was found; every observed start-time gap is 86,400,000 milliseconds. Raw candles, processed volatility and the quality report are stored separately. Perpetual futures are not identical to spot BTC-USD, so conclusions remain market-specific.

The completion rule is especially important for a daily target. A partially formed close would change the newest return, rolling volatility, volume and trade count simultaneously, making the last row incomparable with every earlier completed candle.

The 16 November 2025 cutoff was derived from the original chronological split and then frozen. New candles extend the test end rather than moving its start or becoming training rows. For a same-code comparison, the final pipeline was run on

a slice ending 12 July and the current input ending 19 July. The seven appended days provide a prospective-style stability check within the same market history.

4.2 Feature construction and leakage controls

Returns, activity variables, lags and rolling measures use only information observed by origin t; the target is shifted to $t+1$ afterwards. Exports retain both origin `date` and predicted `target_date`. Missing warm-up rows are removed before the chronological split.

`realised_volatility_30d` and `rolling_return_std_30d` were the same mathematical feature up to floating-point precision. Retaining both would duplicate persistence, split importance and increase selection probability inside a tree. The matching rolling-standard-deviation column is therefore excluded, leaving 25 tabular inputs and eight LSTM inputs.

The final 30-day frame contains 1,195 rows from 11 April 2023 to 18 July 2026. The fixed training portion contains 950 forecast-origin rows ending 15 November 2025; the test contains 245 origins from 16 November 2025 to 18 July 2026, with target dates through 19 July. No random train-test shuffle is used. Tabular scaling is fitted on training only. LSTM scaling is fitted before its chronological internal-validation block, so validation and test distributions cannot influence the scale parameters.

Effective training counts differ by model even though the cutoff is shared. Linear regression and Random Forest use 950 labelled rows. LSTM uses 921 chronological sequences, divided into 783 fitting sequences and 138 internal-validation sequences. GARCH estimates its return process from 993 pre-cutoff returns, including the warm-up history that cannot form a complete feature row. Rolling has no fitted sample. Reporting these counts avoids implying that structurally different models consume identical observations.

The common fairness constraint is therefore information time rather than an artificial equality of row counts: every fitted quantity ends before the same cutoff, and every forecast uses only information available by its origin. The unequal effective counts are retained as a limitation and evidence of how each architecture consumes history, not concealed behind the headline frame size.

GARCH's extra warm-up returns estimate a return process rather than labelled rolling-volatility targets, while LSTM loses early labels when constructing sequences. Recording those differences is more informative than reporting 950 as though it were every model's identical training sample.

4.3 Models

Rolling historical volatility. The benchmark carries today's rolling proxy forward by one day. It has no fitted parameters and directly exploits the overlap between adjacent windows.

GARCH(1,1). A deterministic coarse-to-fine Gaussian maximum-likelihood grid search is combined with variance targeting. The likelihood scores r_t using h_t before r_t^2 updates h_{t+1}, preventing the current shock from entering its own variance. Parameters remain fixed throughout the primary test, but conditional variance updates when a new return becomes observable. Forecasts are aligned by date, and missing or duplicate mappings cause failure.

Lagged linear regression. A linear equation is fitted to standardised inputs with ridge 10^{-8}. Every coefficient is exported. Coefficients describe association on the standardised scale; correlated lagged measures still prevent causal interpretation.

Random Forest. The project-local regressor uses 160 bootstrap trees, maximum depth 7, minimum leaf size 10 and approximately the square root of the available features at each split. Seed 42 makes the forest reproducible. Candidate thresholds are feature quantiles, making this lighter than an optimised library implementation. Both impurity and ten-repeat permutation importance are exported. Permutation importance is calculated after forecasting on the holdout and is used only to interpret the completed model, not to select features or retune it.

LSTM. Thirty-observation sequences feed one LSTM layer with 32 hidden units, a 16-unit ReLU layer and one output. Adam uses learning rate `0.003`, weight decay `1e-5` and batches of 32. The last 15% of training sequences form a chronological validation block for early stopping. The final test does not determine the stopping epoch. The primary network completed 50 epochs, selected epoch 20 and contains 5,921 trainable parameters. Seeds 7, 42 and 101 test optimisation sensitivity.

4.4 Evaluation and decision rules

The primary result is a fixed-cutoff chronological holdout. Model parameters and weights are not refitted for each test day, although every method receives information available at that forecast origin. A supplementary expanding-window design divides the test period into four ordered blocks. The first uses the primary training boundary; each later fold adds earlier test observations to training, refits every fitted model and recalculates scaling. This is block-level refitting, not daily online learning.

Robustness also includes 14- and 30-day targets, two chronological test halves, and low-, medium- and high-volatility terciles defined from realised test targets. The regimes are ex-post diagnostic groups, not real-time classifications. A paired circular moving-block bootstrap resamples blocks of 30 days 2,000 times. Thirty days matches the main target's overlap length and helps retain local dependence (Künsch, 1989). Percentile intervals describe uncertainty under this resampling design; they are not universal significance guarantees.

The research question is answered through evidence rather than an arbitrary weighted score:

Dimension	Evidence used	Decision rule
Accuracy	MAE, MSE and RMSE	RMSE sets the displayed rank; MAE disagreements must be discussed
Robustness	Target windows, halves, folds, regimes, refresh and bootstrap	A claimed advantage should not depend on one boundary or a small set of dates
Interpretability	Formula, parameters, coefficients, importance or recorded architecture	Only explanations actually produced by the pipeline are credited
Practicality	Fit/predict time, dependencies and structural size	Timings are local implementation evidence, not universal speed claims
Reproducibility	Fixed dates, seeds, configuration, metadata and tests	Another run should recover the design and explain any data-driven change

A complex model is justified only if its gain is sufficiently large and stable to compensate for weaker transparency or greater implementation burden.

5. Results

5.1 Primary 30-day target

Rank	Model	MAE	MSE	RMSE	RMSE relative to rolling
1	GARCH(1,1)	0.00047642	0.00000097	0.00098502	-30.994%
2	Lagged linear regression	0.00073861	0.00000196	0.00140087	-1.861%
3	Rolling historical volatility	0.00062843	0.00000204	0.00142744	0.000%
4	LSTM	0.00104907	0.00000304	0.00174351	+22.142%
5	Random Forest	0.00135845	0.00000540	0.00232370	+62.787%

GARCH has the lowest MAE and RMSE. Its `−0.00044242` RMSE difference is a 30.994% reduction from rolling: approximately 0.0985 rather than 0.1427 daily-volatility percentage points.

Linear ranks second by RMSE but has higher MAE than rolling, so its small gain needs uncertainty evidence. LSTM and Random Forest are `+0.00031607` and `+0.00089625` above rolling by RMSE.

After de-duplication, both forest importance methods rank current 30-day volatility, its one-day lag, 30-day mean absolute return and its two-day volatility lag highest, in that order. These are associations, not causes; the forest largely reconstructs persistence more compactly expressed by simpler models.

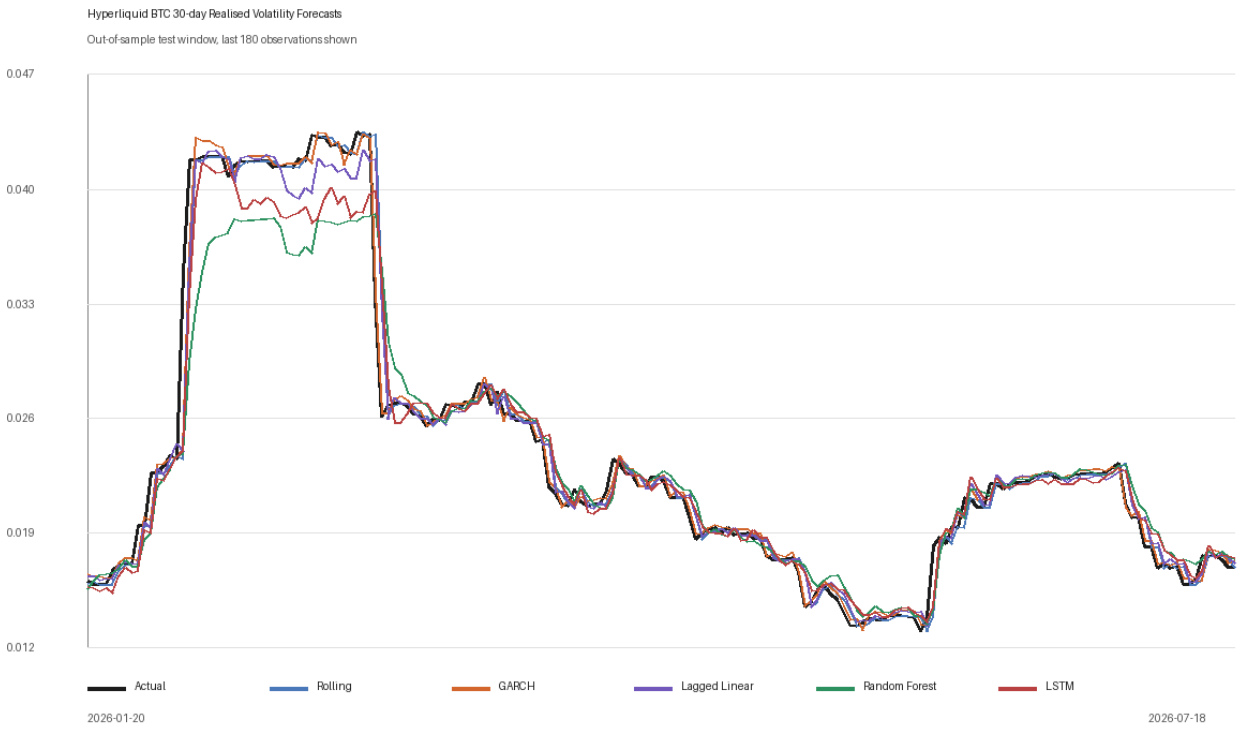

Figure 1. Actual next-day rolling-volatility proxies and forecasts over the displayed test interval. The underlying CSV retains every test observation; the chart limits the visible period for legibility.

Persistent stretches let all forecasts track the target closely, whereas level changes create the largest separation. The chart must therefore be read with regime and bootstrap evidence rather than letting calm periods or one conspicuous miss determine the ranking.

It also illustrates why visual inspection alone is insufficient: several coloured paths overlap for long intervals, but their errors at the relatively few sharp transitions contribute disproportionately to RMSE. MAE, regime bias and resampled differences expose aspects that the compressed plot can hide.

5.2 Target, time and uncertainty robustness

Target window	GARCH RMSE	Linear RMSE	Rolling RMSE	LSTM RMSE	Random Forest RMSE
14 days	0.00178208	0.00260295	0.00276213	0.00364246	0.00405515
30 days	0.00098502	0.00140087	0.00142744	0.00174351	0.00232370

GARCH ranks first at 14 days, 35.482% below rolling. This shorter window has larger errors for every model because each entering or leaving return has greater influence. GARCH also ranks first in both test halves, with RMSE `0.00128466` and `0.00054380`.

Model	Point RMSE difference vs rolling	30-day block-bootstrap interval	Resamples favouring model
GARCH(1,1)	-0.00044242	[-0.00099670, -0.00017158]	100.00%
Lagged linear regression	-0.00002657	[-0.00009605, 0.00003681]	80.45%
LSTM	+0.00031607	[0.00000898, 0.00063980]	2.00%

Model	Point RMSE difference vs rolling	30-day block-bootstrap interval	Resamples favouring model
Random Forest	+0.00089625	[0.00008895, 0.00158205]	0.10%

GARCH's interval remains below zero; linear's crosses zero. Both machine-learning intervals remain above zero: only 2.00% of resamples favour LSTM and 0.10% favour Random Forest. This supports aggregate underperformance rather than a few isolated misses, although resampling one history does not create independent markets.

5.3 Rolling origin, regimes and diagnostics

Concatenated fold RMSE is `0.00098681`, `0.00139934`, `0.00142744`, `0.00203235` and `0.00228159` for GARCH, linear, rolling, LSTM and Random Forest. GARCH is first in every fold, although its RMSE ranges from `0.00040604` to `0.00172036` as difficulty changes.

LSTM is locally competitive: it ranks third in folds 1 and 4 with RMSE `0.00082578` in each, slightly below rolling and Random Forest but not linear. It falls to fourth in folds 2 and 3, so the local gains do not establish aggregate stability.

GARCH's low-, medium- and high-regime RMSE values are `0.00065099`, `0.00058047` and `0.00146368`. Its `−0.00015218` high-regime bias indicates slight average underprediction when risk matters most.

Random Forest OOB RMSE is `0.00131585` across all 950 training rows, versus `0.00232370` chronologically, a `0.00100785` gap. OOB trees can train on later training-period rows, so this diagnoses internal prediction rather than time validation and exposes weaker cross-period generalisation.

Seeds 7, 42 and 101 yield LSTM RMSE `0.00184673`, `0.00174351` and `0.00183721`, selecting epochs 11, 20 and 16. The `0.00010322` range shows optimisation variation, but every run remains above rolling's `0.00142744`.

5.4 Seven-day refresh stability

The later refresh appends completed candles while preserving the 16 November 2025 boundary, testing stability without reallocating earlier test observations.

Model	RMSE using data through 12 July	RMSE using data through 19 July	Rank change
GARCH(1,1)	0.00099309	0.00098502	1 → 1
Lagged linear regression	0.00141843	0.00140087	2 → 2
Rolling historical volatility	0.00144481	0.00142744	3 → 3
LSTM	0.00176625	0.00174351	4 → 4
Random Forest	0.00235585	0.00232370	5 → 5

The added week lowers every RMSE by about 0.8–1.4% but leaves every rank unchanged. It is a procedural stability check, not an independent replication.

5.5 GARCH conversion sensitivity

Conversion from conditional variance	MAE	RMSE
$E[s \mid \mathcal{F}_t]$, 80-point Gauss-Hermite	0.00047642	0.00098502
$\sqrt{E[s^2 \mid \mathcal{F}_t]}$, analytic sensitivity	0.00048149	0.00098447

The analytic sensitivity has mean forecasts `0.00000950` higher and RMSE only `0.00000055` lower. GARCH remains first under both. Quadrature remains primary because it estimates the conditional mean of the evaluated standard-deviation target.

5.6 Practicality and interpretability

Model	Fit time	Predict time	Structural evidence	Interpretation evidence
Rolling	0.000000 s	0.000016 s	0 fitted parameters	Direct persistence rule
Linear regression	0.000236 s	0.000037 s	26 coefficients including intercept	Exported signed coefficients, with collinearity caveat
GARCH(1,1)	1.001552 s	0.020569 s	3 reported parameters	Baseline variance, shock response and persistence
LSTM	5.422232 s	0.010940 s	5,921 trainable parameters	Architecture, loss history and multi-seed evidence
Random Forest	5.412435 s	0.038393 s	17,984 tree nodes	Impurity/permutation importance and OOB diagnostic

GARCH fits in about one second and combines the lowest error with three reported parameters. Each machine-learning fit takes about 5.4 seconds and adds structural and tuning choices without aggregate gain. These local, unequally optimised implementations support only a project-scale decision; parameters, weights and tree nodes are not interchangeable complexity units.

6. Discussion

The evidence answers the research question across its four dimensions rather than through one score. GARCH ranks first by both MAE and RMSE, remains first across both target windows, both test halves, every expanding-window fold and every volatility regime, and its variance dynamics are more directly inspectable than those of the machine-learning models. Rolling is the simplest method and remains a demanding reference. Linear regression is transparent and fast, but its 1.861% RMSE improvement is accompanied by worse MAE and a bootstrap interval crossing zero. Random Forest and LSTM do not deliver a sufficiently stable aggregate gain to compensate for their additional choices and weaker forecast-level explanation.

No arbitrary numerical weight is assigned to transparency or speed. Instead, accuracy establishes whether a candidate deserves consideration, robustness tests whether that gain persists, and interpretability and practicality decide whether remaining complexity is defensible. This prevents one locally winning fold or a marginal timing difference from dominating the full evaluation.

GARCH's performance is theoretically plausible. Conditional variance responds to the latest squared shock while retaining earlier variance through β. Its persistence of `0.88` indicates substantial memory while remaining below the stationary boundary. The rolling-target conversion also gives GARCH a relevant task: 29 returns in the next 30-day window are known, while uncertainty about the entering return is supplied by h_{t+1}. Rolling assumes that the incoming observation produces no predictable update; GARCH replaces that assumption with an estimated variance contribution.

The Gauss-Hermite check qualifies this explanation. The primary expected-standard-deviation forecast is not algebraically identical to the square root of expected variance. Their mean forecasts differ by only `0.00000950`, their RMSE values differ by `0.00000055`, and GARCH remains first under either conversion. This is stronger than either ignoring the distinction or rebuilding the main argument around a numerically negligible sensitivity.

The rolling benchmark's strength is partly built into the target. Tomorrow's window overlaps heavily with today's, so persistence is a mathematical consequence of target construction as well as a market pattern. This makes rolling fair for a one-day operational update, but it may understate the value of richer models for a wholly future, non-overlapping horizon. It also explains why a very small RMSE advantage should not automatically be called practically important.

Linear regression illustrates the difference between ranking and meaningful improvement. Its RMSE is 1.861% below rolling, but its MAE is higher, and its bootstrap interval crosses zero. Calling it definitively better would therefore overstate the evidence. Removing the exactly duplicated rolling-standard-deviation feature makes coefficient evidence less misleading, but remaining lag and window variables are still correlated. Signs are auditable associations, not independent causal effects.

The machine-learning failures and partial successes are both informative. Random Forest's dominant features show that its ensemble spends much of its structure rediscovering persistence. The OOB-to-future gap suggests that relationships transferable within the training period do not transfer equally well across market periods. Neither impurity nor permutation importance explains one individual forecast, and holdout permutation is deliberately post-hoc rather than another tuning stage.

LSTM provides a more nuanced case. It ranks third and slightly beats rolling and Random Forest in expanding-window folds 1 and 4, demonstrating that the architecture is capable of extracting useful sequence information. It does not beat linear regression in either fold, and its aggregate RMSE, high-volatility behaviour, other folds and seed range show that this local advantage is not dependable enough here. This reconciles the current findings with high-frequency studies that favour neural networks: the correct conclusion is not that LSTM never works, but that this daily sample and compact architecture do not supply a robust benefit.

From a risk-management perspective, GARCH is the most defensible tested model because it combines a 30.994% RMSE reduction versus rolling with stable first-place rankings and a short explanation: recent shocks and persistent conditional variance update tomorrow's risk proxy. Its high-regime bias of `−0.00015218` remains important. Overall superiority does not guarantee conservative forecasts during stress. Rolling remains the easiest implementation fallback, while machine learning would require a stable gain from richer data, tuning or a different horizon before its extra decisions became worthwhile.

The fixed-cutoff refresh guards against result drift: recalculating an 80/20 split would change both the data and the experiment. Freezing the start lets seven appended days extend the established holdout. Every rank remains unchanged and every RMSE declines slightly, so the central conclusion survives this short check.

Development audits strengthen the project while revealing how fragile time-series evidence can be. An earlier GARCH result selected forecasts by a reset dataframe index after feature engineering had removed rows; date mapping corrected the alignment. The likelihood had also updated variance with the current squared return before scoring that same return; reversing the order removed look-ahead. Expected sample variance was corrected so the uncertain next return affects both its square and the random sample mean. Later checks excluded open candles, separated forecast and target dates, removed an exactly duplicated feature, froze the evaluation cutoff and measured the standard-deviation conversion approximation. Thirty-nine automated tests now protect the main dates, formulas, diagnostics and exports.

Six limitations remain. First, the study uses one asset and exchange. Secondly, the target is a daily rolling proxy rather than high-frequency realised variance. Thirdly, folds, regimes and refreshes all belong to one market history. Fourthly, GARCH uses a Gaussian likelihood despite heavy-tailed cryptocurrency returns. Fifthly, Random Forest and LSTM tuning is intentionally limited and not nested inside a separate chronological search. Finally, no portfolio, transaction-cost or Value-at-Risk exercise converts forecast differences into financial outcomes. The project can judge forecast error and practical qualities, but it does not establish profitability or capital adequacy.

A stronger extension would reserve another untouched future period, construct realised variance from intraday returns, test Student-t or asymmetric GARCH, and tune machine learning within nested chronological validation. A hybrid could use GARCH variance as an input to a nonlinear model. Sentiment is another possible addition: Brauneis and Sahiner (2026) find nonlinear sentiment effects, although results vary by coin and Bitcoin is an important exception. Extensions should be introduced separately so the source of any improvement remains identifiable.

7. Conclusion

This project asked whether Random Forest and LSTM improve on rolling historical volatility and GARCH(1,1) when Bitcoin volatility forecasts are judged by accuracy, interpretability, computational practicality and robustness. Under the tested Hyperliquid daily-data design, GARCH records the strongest balance. Its primary RMSE is `0.00098502`, its improvement over rolling is 30.994%, and it ranks first at both target windows, in both test halves, all four expanding-window folds and all three volatility regimes. The fixed-cutoff refresh and target-conversion sensitivity leave that conclusion unchanged.

Linear regression remains close to rolling but its improvement is uncertain because MAE is worse and the bootstrap interval crosses zero. LSTM is competitive in limited contexts yet lacks aggregate stability, while Random Forest's future-test error and structural cost outweigh its nonlinear flexibility. The conclusion is narrow rather than ideological: GARCH is strongest *in this experiment*. Published work shows that richer high-frequency data and more advanced architectures can favour machine learning. Here, complexity does not earn its place through a sufficiently clear, stable and context-relevant gain.

References

Bollerslev, T. (1986) 'Generalized autoregressive conditional heteroskedasticity', *Journal of Econometrics*, 31(3), pp. 307–327. Available at: https://doi.org/10.1016/0304-4076(86)90063-1.

Brauneis, A. and Sahiner, M. (2026) 'Crypto volatility forecasting: Mounting a HAR, sentiment, and machine learning horserace', *Asia-Pacific Financial Markets*, 33, pp. 379–411. Available at: https://doi.org/10.1007/s10690-024-09510-6.

Breiman, L. (2001) 'Random forests', *Machine Learning*, 45, pp. 5–32. Available at: https://doi.org/10.1023/A:1010933404324.

Catania, L., Grassi, S. and Ravazzolo, F. (2019) 'Forecasting cryptocurrencies under model and parameter instability', *International Journal of Forecasting*, 35(2), pp. 485–501. Available at: https://doi.org/10.1016/j.ijforecast.2018.09.005.

Dudek, G., Fiszeder, P., Kobus, P. and Orzeszko, W. (2024) 'Forecasting cryptocurrencies volatility using statistical and machine learning methods: A comparative study', *Applied Soft Computing*, 151, 111132. Available at: https://doi.org/10.1016/j.asoc.2023.111132.

Hansen, P.R. and Lunde, A. (2005) 'A forecast comparison of volatility models: Does anything beat a GARCH(1,1)?', *Journal of Applied Econometrics*, 20(7), pp. 873–889. Available at: https://doi.org/10.1002/jae.800.

Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735–1780. Available at: https://doi.org/10.1162/neco.1997.9.8.1735.

Huang, Z.-C., Sangiorgi, I. and Urquhart, A. (2024) 'Forecasting Bitcoin volatility using machine learning techniques', *Journal of International Financial Markets, Institutions and Money*, 97, 102064. Available at: https://doi.org/10.1016/j.intfin.2024.102064.

Hyperliquid (2026) 'Info endpoint', *Hyperliquid Docs*. Available at: https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api/info-endpoint (Accessed: 20 July 2026).

Katsiampa, P. (2017) 'Volatility estimation for Bitcoin: A comparison of GARCH models', *Economics Letters*, 158, pp. 3–6. Available at: https://doi.org/10.1016/j.econlet.2017.06.023.

Künsch, H.R. (1989) 'The jackknife and the bootstrap for general stationary observations', *The Annals of Statistics*, 17(3), pp. 1217–1241. Available at: https://doi.org/10.1214/aos/1176347265.

Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems*, 30. Available at: https://arxiv.org/abs/1705.07874.

Molnar, C. (2025) *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 3rd edn. Available at: https://christophm.github.io/interpretable-ml-book/ (Accessed: 20 July 2026).

Patton, A.J. (2011) 'Volatility forecast comparison using imperfect volatility proxies', *Journal of Econometrics*, 160(1), pp. 246–256. Available at: https://doi.org/10.1016/j.jeconom.2010.03.034.

Shen, Z., Wan, Q. and Leatham, D.J. (2021) 'Bitcoin return volatility forecasting: A comparative study between GARCH and RNN', *Journal of Risk and Financial Management*, 14(7), 337. Available at: https://doi.org/10.3390/jrfm14070337.

Zahid, M., Iqbal, F. and Koutmos, D. (2022) 'Forecasting Bitcoin volatility using hybrid GARCH models with machine learning', *Risks*, 10(12), 237. Available at: https://doi.org/10.3390/risks10120237.